Open-loop regeling van een aeropendulum

Leerdoelen

- ❒ Je kunt overweg met de Arduino IDE en begrijpt de werking van een H-brug, PWM en een hoeksensor.
- ❒ Je begrijpt de basis van I^2C communicatie en weet de functie van SDA en SCL.
- ❒ Je kunt vanuit Python de Arduino aansturen, meetgegevens opslaan en deze vervolgens analyseren en visualiseren.
- ❒ Je kunt de dynamische respons van een simpel open-loop systeem experimenteel bepalen.

Veiligheid

1. Houd tijdens de metingen voldoende afstand van de draaiende propeller en raak deze nooit aan wanneer de voeding is ingeschakeld.
2. Controleer vóór iedere meting of het aeropendulum vrij kan bewegen en of zich geen objecten binnen het bewegingsbereik bevinden.
3. De hoeksensor werkt op **3,3 V**. Sluit deze nooit rechtstreeks aan op de 5 V van de Arduino.
4. Twijfel je over een handeling of apparaat? Vraag dit altijd aan de practicumbegeleider.

Introductie

Regeltechniek speelt een belangrijke rol binnen de technische natuurkunde. Veel technische systemen, zoals drones, robots, satellieten, productieprocessen en autonome voertuigen, moeten automatisch bestuurd worden om een gewenste positie of toestand te bereiken. Voordat een regelsysteem ontworpen kan worden, is het belangrijk om eerst het gedrag van het systeem zelf goed te begrijpen. Dit gebeurt vaak door het systeem in open-loop te bestuderen.

Bij een open-loop systeem wordt een ingangssignaal aangeboden zonder dat de uitgang wordt teruggekoppeld naar de regeling. De besturing corrigeert dus niet voor verstoringen of afwijkingen. Hoewel deze manier van regelen relatief eenvoudig is, vormt zij de basis voor vrijwel alle moderne regelsystemen. Door de respons van een systeem op verschillende ingangssignalen te meten, kunnen eigenschappen zoals de gevoeligheid, tijdconstante, demping en niet-lineariteiten worden bepaald.

Een eenvoudig voorbeeld van open-loop regeling is een magnetron. Je stelt bijvoorbeeld een vermogen van 800 W en een tijd van 2 minuten in. De magnetron voert deze instelling uit, maar meet niet of het eten daadwerkelijk de gewenste temperatuur heeft bereikt. Is het eten na 2 minuten nog te koud? Dan corrigeert de magnetron dit niet automatisch. De uitgang (temperatuur van het eten) wordt namelijk niet teruggekoppeld naar de ingang (vermogen en verwarmingstijd).

In dit practicum onderzoek je het dynamische gedrag van een aeropendulum. Dit systeem bestaat uit een arm die om een draaipunt kan roteren en wordt aangedreven door een elektrische propeller. De propeller levert een stuwkracht die de arm omhoog beweegt, terwijl de zwaartekracht de arm terug naar beneden probeert te brengen. Door de motorspanning stapsgewijs te variëren ontstaat een evenwicht tussen deze twee krachten, waardoor verschillende stationaire hoeken en dynamische responsen kunnen worden bestudeerd.

Tijdens het practicum meet je de hoek van het aeropendulum als functie van de tijd voor verschillende motorspanningen. De meetgegevens worden vervolgens in geanalyseerd om de dynamische eigenschappen van het systeem te bepalen en te vergelijken met een eenvoudig fysisch model. De kennis die in dit practicum wordt opgedaan vormt de basis voor een vervolgpracticum, waarin hetzelfde systeem met behulp van terugkoppeling (*closed-loop control*) automatisch zal worden geregeld.

Theorie

Een aeropendulum bestaat uit een draaibare arm met aan het uiteinde een elektromotor en propeller. De propeller levert een stuwkracht waardoor de arm omhoog beweegt. De zwaartekracht werkt deze beweging tegen. Door het vermogen van de motor te variëren kan de positie van het aeropendulum worden beïnvloed.

In dit practicum wordt het systeem in open-loop onderzocht. Dit betekent dat het vermogen van de motor vooraf wordt ingesteld en niet automatisch wordt aangepast op basis van de gemeten hoek. Voor iedere instelling wordt vervolgens onderzocht hoe het aeropendulum reageert.

De propeller levert een stuwkracht F_T op een afstand l_T van het draaipunt. Het bijbehorende moment is

$$M_T = l_T F_T. \tag{1}$$

Tegelijkertijd oefent de zwaartekracht een moment uit op het systeem. Wanneer het zwaartepunt zich op een afstand l van het draaipunt bevindt, is dit moment

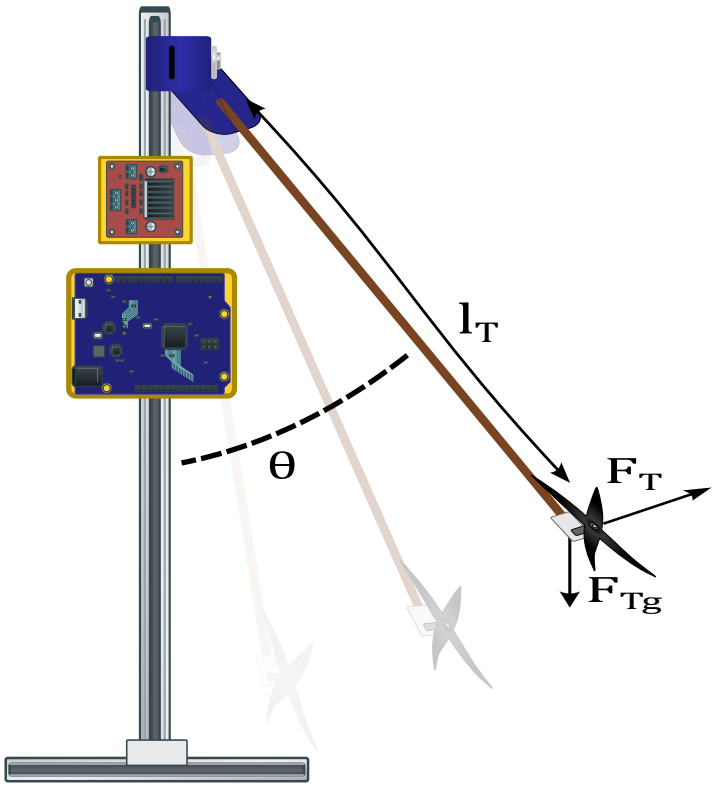
$$M_g = mgl \sin\theta, \tag{2}$$

waarbij m de massa van het bewegende systeem is, g de zwaartekrachtsversnelling en θ de hoek van de arm.

Volgens de tweede wet van Newton voor rotatie wordt de beweging bepaald door de som van deze momenten:

$$J\ddot{\theta} = l_T F_T - mgl \sin\theta, \tag{3}$$

waarbij J het massatraagheidsmoment van het aeropendulum rond het draaipunt is.



Figuur 1: Schematische weergave van het aeropendulum en de relevante krachten.

De stuwkracht van de propeller wordt bepaald door het vermogen dat aan de motor wordt geleverd. Het elektrisch ingangsvermogen is

$$P_{\text{el}} = UI, \tag{4}$$

met U de spanning over de motor en I de stroom door de motor. Slechts een deel van dit vermogen wordt door de motor en propeller omgezet in mechanisch vermogen. Met een rendement η kan dit worden geschreven als

$$P_{\text{motor}} = \eta UI. \tag{5}$$

De stuwkracht kan daarom worden beschouwd als een functie van het geleverde vermogen,

$$F_T = f(P_{\text{motor}}). \tag{6}$$

De precieze vorm van deze relatie hangt af van onder andere de motor en de propeller en hoeft niet lineair te zijn.
Wanneer het aeropendulum uiteindelijk tot stilstand komt, is de hoekversnelling nul. Het moment van de propeller is dan gelijk aan het moment van de zwaartekracht:

$$l_T F_T = mgl \sin\theta. \tag{7}$$

Hieruit volgt

$$\theta = \arcsin\left(\frac{l_T F_T}{mgl}\right). \tag{8}$$

Een groter motorvermogen leidt in het algemeen tot een grotere stuwkracht en daarmee tot een grotere evenwichtshoek. Tijdens het practicum wordt deze relatie experimenteel onderzocht door het motorvermogen te variëren en de resulterende hoek van het aeropendulum te meten.

Componenten

Het aeropendulum bestaat uit verschillende elektronische en mechanische componenten. De Arduino stuurt de motor aan via een H-brug en leest tegelijkertijd de hoek van het pendulum uit met een magnetische hoeksensor. De aansluittabel bij opdracht 0 geeft een overzicht van de gebruikte aansluitingen.

Arduino Uno R4 Minima

Een Arduino is een programmeerbaar microcontroller waarmee elektronische componenten kunnen worden aangestuurd en sensoren kunnen worden uitgelezen. In dit practicum wordt een **Arduino Uno R4 Minima** gebruikt.
De Arduino voert continu een programma uit waarin twee taken centraal staan: het aansturen van de elektromotor en het meten van de hoek van het aeropendulum. Hiervoor beschikt de Arduino onder andere over digitale aansluitingen, analoge ingangen en verschillende communicatieprotocollen.
In deze opstelling worden de digitale pinnen D9, D10 en D11 gebruikt voor de aansturing van de motor. De AS5600-hoeksensor communiceert via de pinnen A4 en A5 met de Arduino.

H-brug

Een elektromotor kan niet rechtstreeks vanuit een digitale uitgang van de Arduino worden gevoed. De motor vraagt aanzienlijk meer stroom dan een uitgang van de microcontroller kan leveren. De Arduino levert daarom alleen het *stuursignaal*; het elektrische vermogen voor de motor wordt geleverd door de externe voeding.

Tussen de Arduino en de motor bevindt zich daarom een **L298N H-brug**. Deze schakeling kan de externe voedingsspanning naar de motor schakelen. De naam H-brug komt van de configuratie van de interne schakelaars, die schematisch de vorm van de letter H heeft.

Door verschillende schakelaars te openen en sluiten kan de stroom door de motor in beide richtingen lopen. Daarmee kan ook de draairichting van de motor worden veranderd. Hiervoor gebruikt de L298N de ingangen IN1 en IN2. In onze opstelling worden deze aangestuurd door D10 en D11 van de Arduino.

Pulse Width Modulation

Naast de draairichting willen we ook kunnen regelen hoeveel vermogen de motor ontvangt. Hiervoor wordt **Pulse Width Modulation (PWM)** gebruikt.

Bij PWM wordt de voedingsspanning niet continu veranderd. In plaats daarvan wordt de spanning snel aan- en uitgeschakeld. De verhouding tussen de tijd dat het signaal aan staat en de totale periodetijd noemen we de *duty cycle* D:

$$D = \frac{t_{\text{aan}}}{T}. \tag{9}$$

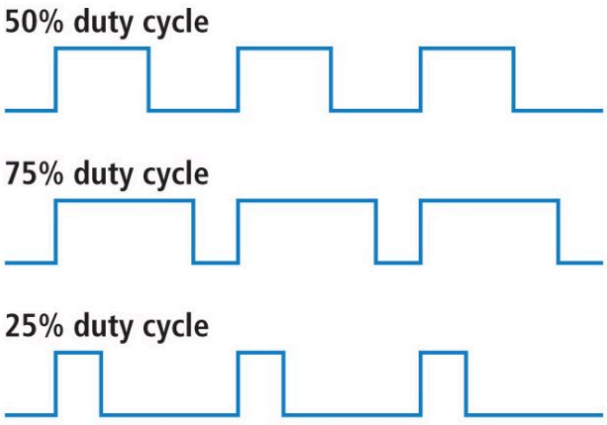
Een duty cycle van 100% betekent dat de motor continu wordt aangestuurd. Bij 50% staat de spanning gedurende de helft van iedere periode aan en bij 0% staat de motor volledig uit.

Voor een ideale PWM-schakeling is de gemiddelde spanning over één periode

$$U_{\text{gem}} = D\,U_{\text{voeding}}. \tag{10}$$

De motor reageert door zijn mechanische en elektrische traagheid niet afzonderlijk op iedere snelle puls. Daardoor kan met de duty cycle het gedrag van de motor worden geregeld.

In onze opstelling genereert pin D9 van de Arduino het PWM-signaal. Dit signaal wordt aangesloten op de ENA-ingang van de L298N. De Arduino bepaalt dus wanneer de H-brug de motor inschakelt, terwijl het daadwerkelijke motorvermogen uit de externe voeding komt.



Figuur 2: Voorbeeld van PWM-signalen met verschillende duty cycles. De amplitude blijft gelijk, maar de verhouding tussen de aan- en uittijd verandert.

AS5600-hoeksensor

De hoek van het aeropendulum wordt gemeten met een **AS5600 magnetische hoeksensor**. Op de draaiende as van het aeropendulum wordt een magneet geplaatst. De sensor meet de richting van het magnetische veld en kan daaruit de hoek van de as bepalen. Omdat er geen mechanisch contact nodig is tussen de sensor en de draaiende as, treedt hierbij vrijwel geen extra mechanische weerstand op.

De AS5600 heeft een resolutie van 12 bit. Eén volledige omwenteling wordt daardoor verdeeld in

$$2^{12} = 4096 \tag{11}$$

mogelijke waarden. Een ruwe sensorwaarde kan worden omgerekend naar een hoek volgens

$$\theta = \frac{\text{sensorwaarde}}{4096}\, 360^\circ. \tag{12}$$

De sensor wordt digitaal uitgelezen via het **I²C-protocol**, zie Appendix C voor uitleg over deze communicatieprotocol. Hiervoor zijn twee verbindingen nodig: SDA (*Serial Data*) voor het versturen van gegevens en SCL (*Serial Clock*) voor het kloksignaal. In deze opstelling is SDA aangesloten op A4 en SCL op A5 van de Arduino. Het volledige schakelschema is te vinden in Appendix B

De positie van de magneet ten opzichte van de sensor is belangrijk. Wanneer de magneet te ver van de sensor staat of niet goed boven het midden van de sensor is geplaatst, kan de hoekmeting onbetrouwbaar worden. Controleer daarom voor iedere meting of de sensor correct functioneert. Wanneer de arm langzaam met de hand wordt bewogen, moet de gemeten hoek vloeiend veranderen. Een constante waarde, grote sprongen of een foutmelding kan betekenen dat de magneet niet correct ten opzichte van de sensor staat.

Voorbereiding

1. Installeer de Arduino IDE en de Python-omgeving op je laptop. Controleer of de Arduino Uno R4 Minima wordt herkend en dat de juiste poort kan worden geselecteerd **Link naar Arduino IDE**.
2. Installeer de bibliotheek `AS5600` in de Arduino IDE. Controleer of de bibliotheek correct is geïnstalleerd door een voorbeeldsketch te openen en te uploaden naar de Arduino.
3. Clone or download the github repository of the practicum guide. Controleer of de bestanden correct zijn gedownload en dat je toegang hebt tot de arduino- en pythonbestanden in de map. **Link naar github repository**
4. Bekijk de geometrie van het aeropendulum. Onder welke voorwaarde geldt $l = l_T$? Leg in woorden uit wat deze aanname fysisch betekent.
5. In de stationaire toestand staat de arm stil en zijn de momenten rond het draaipunt in evenwicht. Laat zien dat dan geldt:
$$l_T F_T = mgl \sin\theta. \tag{13}$$
6. De motor zet niet al het opgenomen elektrische vermogen om in mechanisch vermogen. Neem een rendement van $\eta = 0.70$ aan:
$$P_{\text{mech}} = \eta U I. \tag{14}$$
Bereken bij een voedingsspanning van $U = 5$ V het mechanisch vermogen voor een aantal verschillende motorstromen.
7. Neem aan dat de stuwkracht van de propeller evenredig is met het mechanisch vermogen:
$$F_T = C P_{\text{mech}}, \tag{15}$$
waarbij C een constante is die de eigenschappen van de motor en propeller beschrijft. Combineer de bovenstaande vergelijkingen en leid een uitdrukking af voor de stationaire hoek θ als functie van de motorstroom I.
8. Kies realistische waarden voor m, l, l_T en C. Licht kort toe waarom je deze waarden hebt gekozen. Maak vervolgens in Desmos een grafiek van θ als functie van I.
9. Voorspel op basis van je grafiek wat er gebeurt wanneer de motorstroom wordt verhoogd. Verwacht je dat het eenvoudige model voor alle motorstromen even goed werkt? Leg kort uit.

Experiment

Werk de opdrachten in onderstaande volgorde uit. Open iedere sketch vanuit zijn eigen gelijknamige map onder `Arduino/OpenLoop`. Selecteer in de Arduino IDE de Arduino Uno R4 Minima en de juiste poort. Installeer de bibliotheek `AS5600` van Rob Tillaart. Alle drie de scripts gebruiken 115200 baud.

Opdracht 0: opbouw van de opstelling en aansluitschema

1. Zet de Arduino Uno R4 Minima, AS5600-hoeksensor, L298N H-brug, motor en externe voeding klaar. Bekijk hoe de arm en de magneet bij het draaipunt zijn gemonteerd. Controleer of de arm vrij kan bewegen.
2. Bouw de opstelling op met uitgeschakelde externe voeding en losgekoppelde USB-kabel. Sluit de hoeksensor aan volgens tabel 1. Controleer per draad de Arduino-pin, draadkleur en aansluiting op de component. De H-brug gaan we in de volgende opdracht aansluiten.
3. Laat de aansluiting van motor en de externe voeding op de H-brug aansluiting controleren door de practicumbegeleider. Spreek de voedingsspanning en stroomlimiet af en noteer deze in je labjournaal.
4. Laat de complete opstelling controleren voordat je USB aansluit. Houd de motorvoeding uitgeschakeld: in opdracht 1 test je eerst alleen de hoeksensor met `Stap_1_Sensor.ino`.

Tabel 1: Aansluitingen van de Arduino naar de AS5600-hoeksensor.

Van Arduino	Draadkleur	Naar component	Aansluiting
GND (Analog)	Zwart	Hoeksensor	GND
3.3 V	Rood	Hoeksensor	VCC
A4	Groen	Hoeksensor	SDA
A5	Wit	Hoeksensor	SCL

Opdracht 1: hoeksensor uittesten met stap 1

1. Gebruik de in opdracht 0 opgebouwde en gecontroleerde opstelling. Laat de motorvoeding uitgeschakeld en sluit alleen USB aan op de Arduino. Open `Arduino/OpenLoop/Stap_1_Sensor/Stap_1_Sensor.ino`: deze sketch leest uitsluitend de hoeksensor uit en stuurt de motor niet aan. Controleer in de code of je de juiste poorten uitleest.
2. Upload het script en open Serial Monitor of Serial Plotter op 115200 baud. Het script geeft iedere 100 ms een regel zoals `Hoek:90.000` weer.
3. Bekijk de omzetting `sensor.readAngle() * 360.0 / 4096.0` in de code. Leg uit waarom door 4096 wordt gedeeld en bepaal hoeveel graden een sensorstap voorstelt.
4. Beweeg de arm langzaam met de hand. Controleer of de gemeten hoek vloeiend verandert en noteer de sensorhoek bij de verticale ruststand en bij enkele bekende armstanden. Onderzoek in welke richting de sensorhoek toeneemt en waar de overgang tussen 359 en 0 graden ligt.
5. Bewaar je waarnemingen in het labjournaal. Laat een constante waarde, onverwachte sprongen of een onjuiste hoek eerst controleren voordat je de motor aansluit. Deze scripts controleren de sensorstatus niet automatisch.

Opdracht 2: vaste PWM en stationaire hoek

1. Sluit de voeding en de H-brug aan volgens het schema Tabel 2.
2. Open `Stap_2_PWM_en_sensor/Stap_2_PWM_en_sensor.ino`. Schakel de voeding uit en controleer de in opdracht 0 aangesloten H-brug en motor. Laat de motorvoeding en de ENA-aansluiting controleren door de begeleider.
3. Zoek `pwm_waarde` en de drie motorpinnen op in de code. Leg uit hoe D9 het PWM-signaal levert en hoe D10 en D11 de draairichting vastleggen. Bereken de duty cycle bij de ingestelde waarde met $D = \frac{\text{pwm_waarde}}{255}$.
4. Spreek met de begeleider een veilige voedingsspanning en stroomlimiet af, dit is typisch rond de 5V. Het codebereik voor de PWM (0-255) is geen vastgesteld veilig werkbereik, dit hangt af van de voedingsspanning. Verder volgt de stroom uit de belasting van de motor en de propeller.
5. Kies vervolgens een PWM-waarde en upload opnieuw met de motorvoeding uit. Schakel daarna de motorvoeding in. De sketch stuurt de motor direct na uploaden of resetten aan met `pwm_waarde`[1]. Er is geen startcommando, automatische meetduur of softwarematige stop in dit script; stop door de motorvoeding uit te schakelen.
6. Bekijk de hoek in Serial Monitor of Serial Plotter. Wacht tot de arm een stationaire positie bereikt, of noteer dat dit niet gebeurt. Noteer de PWM-waarde, duty cycle, stationaire werkhoek, voedingsspanning en motorstroom.
7. Herhaal dit voor meerdere PWM-waarden, noteer de maximale hoek en de bijbehorende motorstroom.
8. Maak een grafiek van de gemiddelde stationaire werkhoek tegen de PWM-waarde, met foutbalken op basis van de herhalingen. Bepaal de dode zone en onderzoek of de relatie lineair is. Dit script verstuurt alleen `Hoek:`-regels, zonder tijdstempels of CSV-kop.

Tabel 2: Aansluitingen van de Arduino naar de L298N H-brug.

Van Arduino	Draadkleur	Naar component	Aansluiting
D9 (digitaal, PWM)	Paars	H-brug	ENA
D10 (digitaal)	Oranje	H-brug	IN1
D11 (digitaal)	Geel	H-brug	IN2

[1] Je kunt de Arduino resetten (bij bijvoorbeeld een storign) door de resetknop op de Arduino één keer in te drukken. Ook het openen van een seriele verbinding kan een reset veroorzaken.

Opdracht 3: aansturen en meten met Python

1. Open `Stap_3_Python_aansturing/Stap_3_Python_aansturing.ino` en upload met uitgeschakelde motorvoeding. De motor begint bij deze sketch met PWM 0. Sluit Serial Monitor en Serial Plotter voordat Python dezelfde poort opent.
2. Installeer in je Python-omgeving `pyserial`, `pandas` en `matplotlib`.
3. Open Practicum2.2/open_loop/script.ipynb en controleer of je de eerste cel kunt runnen.
4. Test de nulhoek van de sensor bij PWM 0. Controleer of de gemeten hoek vloeiend verandert wanneer je de arm langzaam met de hand beweegt. Noteer de beginhoek in het labjournaal.
5. Sla iedere proef op onder een eigen bestandsnaam. De Arduino verstuurt meetregels als `tijd_in_ms,hoek_in_graden`, zonder kopregel. De tijd loopt vanaf het starten van de Arduino.
6. Herhaal de metingen minimaal drie keer per PWM-instelling. Noteer de PWM-waarde, voedingsspanning, motorstroom, beginhoek en hoekconversie apart. Maak grafieken van de werkhoek tegen de tijd en bepaal per proef de gemiddelde stationaire hoek en standaarddeviatie over een onderbouwd gekozen tijdvenster.
7. Vergelijk de resultaten met de voorspellingen uit de voorbereiding. Bespreek eventuele afwijkingen en mogelijke oorzaken.
8. Maak een fit op basis van de voorbereiding, waarom lukt dat (niet)?

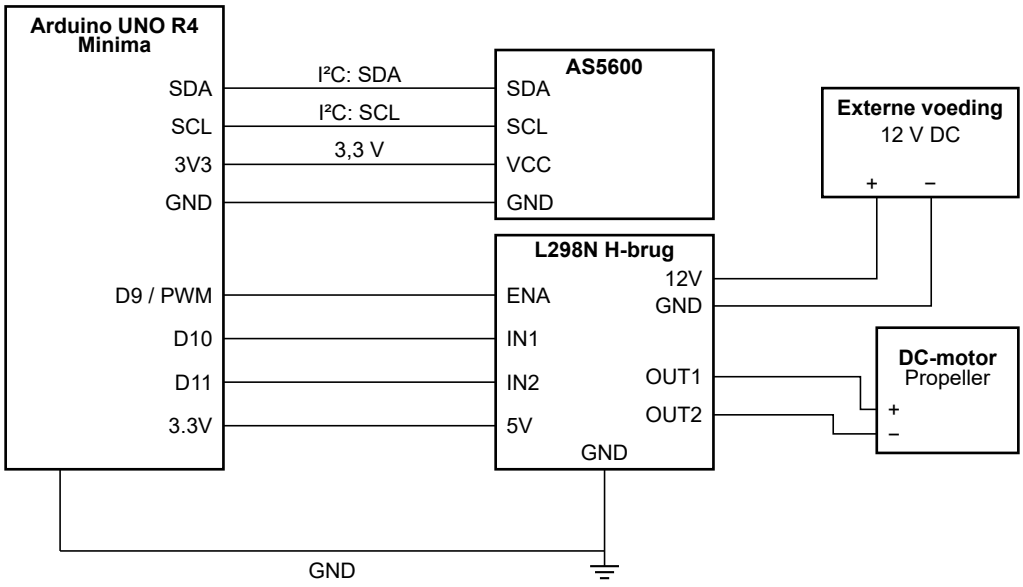
Afronding

1. Laat de meetresultaten controleren door de practicumbegeleider.
2. Voeg alle grafieken toe aan je digitale labjournaal en geef iedere figuur een duidelijk onderschrift.
3. Vul de sectie **Resultaten en Discussie** aan. Beschrijf hierin de relatie tussen de Arduino-uitgang en de stationaire hoek en vergelijk de experimentele resultaten met het theoretische model.
4. Reflecteer op de beperkingen van een open-loop systeem en beschrijf waarom terugkoppeling in veel technische toepassingen gewenst is.
5. Breek de opstelling af en ruim de werkplek op. Controleer of alle componenten compleet zijn en berg deze op in de daarvoor bestemde bakken.

Appendix A: Materialen Lijst Practica

- Arduino Uno R4 Minima;
- L298N H-brug;
- AS5600 magnetische hoeksensor;
- Elektromotor met propeller;
- Externe voeding;
- USB-C kabel en koppelstuk;
- Platform met paal en 3D geprint aansluitstuk, arm met draaipunt en magneet;
- Diverse aansluitdraden (Male-Male, Male-Female, Female-Female);
- Laptop of computer met Arduino IDE en Python 3.11+;

Appendix B: Elektrisch schema aeropendulum



Figuur 3: Elektrisch aansluitschema van de aeropendulumopstelling. De Arduino UNO R4 Minima leest de hoekpositie uit via de AS5600-sensor (I²C) en stuurt de elektromotor aan via de L298N H-brug (PWM). De motor wordt gevoed door een externe 12V-voeding. Alle componenten delen een gemeenschappelijke massa (GND).

Appendix C: Communicatieprotocollen

Bij digitale communicatie wordt informatie verzonden in de vorm van bits: nullen (0) en enen (1). Een groep van 8 bits noemen we een byte.
Er bestaan twee manieren om deze bits te versturen:

- **Seriële communicatie:** de bits worden één voor één, achter elkaar, over een datalijn verzonden.
- **Parallelle communicatie:** meerdere bits worden tegelijkertijd over afzonderlijke datalijnen verzonden.

Stel dat we de byte `10110010` willen versturen.
Bij seriële communicatie worden de acht bits achtereenvolgens over dezelfde datalijn verzonden. Bij parallelle communicatie kunnen alle acht bits tegelijkertijd via acht afzonderlijke datalijnen worden

verstuurd. Seriële communicatie vereist doorgaans minder aansluitdraden en is daardoor bijzonder geschikt voor communicatie tussen elektronische componenten.

Communicatieprotocollen

Een communicatieprotocol is een verzameling afspraken waarmee elektronische apparaten gegevens kunnen uitwisselen. Het protocol beschrijft onder andere hoe informatie wordt verzonden, hoe apparaten elkaar herkennen en hoe wordt gecontroleerd of gegevens correct zijn ontvangen. Het I²C-protocol maakt gebruik van seriële communicatie. De bits worden één voor één via de SDA-lijn verzonden, terwijl het kloksignaal op de SCL-lijn bepaalt wanneer de bits worden overgedragen.
Bij microcontrollers worden verschillende communicatieprotocollen gebruikt:

- **UART (Universal Asynchronous Receiver/Transmitter):** seriële communicatie tussen twee apparaten via een zend- (TX) en ontvangstlijn (RX). Wordt bijvoorbeeld gebruikt voor communicatie tussen een microcontroller en een computer.
- **SPI (Serial Peripheral Interface):** een relatief snel serieel protocol waarbij een controller afzonderlijke apparaten selecteert. Wordt onder andere gebruikt voor displays en geheugenchips.
- **I²C (Inter-Integrated Circuit):** een serieel protocol waarmee meerdere apparaten via slechts twee signaallijnen met elkaar kunnen communiceren.

In dit practicum gebruiken we I²C om de Raspberry Pi Pico te laten communiceren met de digitaal-analoogomzetter (MCP4725) en de stroom- en spanningssensor (INA226).

Het I²C-protocol

I²C (uitgesproken als **I-squared-C**) is een communicatieprotocol dat oorspronkelijk door Philips is ontwikkeld. Het maakt het mogelijk om meerdere geïntegreerde schakelingen (IC's) aan te sluiten op dezelfde communicatiebus.

De twee signaallijnen

I²C maakt gebruik van twee signaallijnen:

- **SDA (Serial Data):** transporteert de gegevens tussen de apparaten.
- **SCL (Serial Clock):** transporteert het kloksignaal waarmee de communicatie wordt gesynchroniseerd.

Daarnaast moeten de aangesloten apparaten een gemeenschappelijke elektrische referentie (GND) hebben. De lijnen SDA en SCL worden via zogenaamde pull-upweerstanden verbonden met de voedingsspanning van de communicatiebus, bijvoorbeeld 3,3 V. Apparaten trekken de lijnen actief naar een lage spanning (logische 0), maar sturen normaal gesproken geen hoge spanning uit. Wanneer geen enkel apparaat een lijn laag trekt, zorgt de pull-upweerstand voor een logische 1.
Dit wordt een open-drain-configuratie genoemd. Hierdoor kunnen meerdere apparaten veilig dezelfde signaallijnen gebruiken.
Bij de meeste microcontrollers (Arduino & Raspberry Pi Pico) zijn pull-upweerstanden geintegreerd binnen de PCB. Echter, extra pull-upweerstanden kan voor stabiliteit geen kwaad. Afhankelijk van de configuratie kunnen soms ook juist pull-downweerstanden nodig zijn.

Controller en targets

Op een I²C-bus bevinden zich één of meerdere controllers en één of meerdere targets.

- **Controller (mother):** initieert de communicatie en genereert het kloksignaal.
- **Target (slave):** reageert wanneer de controller zijn adres aanspreekt.

In ons practicum is de Raspberry Pi Pico de controller. De MCP4725 en INA226 zijn targets.
Beide IC's zijn op dezelfde SDA- en SCL-lijnen aangesloten. De controller bepaalt met welk apparaat wordt gecommuniceerd door het bijbehorende adres te versturen.

Adressering

Elk target heeft een I²C-adres (genoteerd als bijvoorbeeld `0x40` of `0x30`). Meestal wordt hiervoor een adres van 7 bits gebruikt. Daarmee zijn theoretisch 2^7 = 128 verschillende adreswaarden mogelijk, hoewel een deel hiervan gereserveerd is.
Voorbeelden van adressen zijn:

- MCP4725: vaak `0x60`;
- INA226: standaard `0x40`.

De notatie `0x` betekent dat het getal hexadecimaal is weergegeven.
Een hexadecimaal cijfer vertegenwoordigt vier bits. Het adres `0x40` komt bijvoorbeeld overeen met de binaire waarde `1000000` (7 bits)[2] .
Op dezelfde bus[3] kunnen meerdere apparaten worden aangesloten, zolang hun adressen niet conflicteren.

Hoe verloopt de communicatie?

Een typische I²C-transactie bestaat uit de volgende stappen:

1. **START-conditie:** de controller trekt SDA van hoog naar laag terwijl SCL hoog is. Hiermee begint de communicatie.
2. **Adres:** de controller verstuurt het 7-bits adres van het gewenste apparaat.
3. **Lees-/schrijfbit:** één extra bit geeft aan of de controller gegevens wil lezen (1) of schrijven (0).
4. **Bevestiging (ACK):** het aangesproken apparaat trekt tijdens de negende klokpuls SDA laag om te bevestigen dat het de adressering heeft ontvangen.
5. **Gegevens:** de apparaten wisselen één of meerdere bytes uit. Iedere byte bestaat uit 8 bits en wordt gevolgd door een ACK- of NACK-bit.
6. **STOP-conditie:** de controller laat SDA van laag naar hoog gaan terwijl SCL hoog is. De communicatie is hiermee beëindigd.

Bij I²C worden de bits meestal verzonden met het meest significante bit (**MSB**) eerst.
Tijdens normale gegevensoverdracht verandert SDA wanneer SCL laag is. Terwijl SCL hoog is, blijft SDA stabiel. Uitzonderingen zijn de START- en STOP-condities.

Kloksnelheid

De snelheid van I²C wordt bepaald door de frequentie van het kloksignaal op SCL.
Veelgebruikte snelheden zijn:

- **Standard-mode:** maximaal 100 kHz;
- **Fast-mode:** maximaal 400 kHz;
- **Fast-mode Plus:** maximaal 1 MHz.

Niet ieder apparaat ondersteunt alle snelheden. De gekozen kloksnelheid moet daarom worden ondersteund door alle apparaten waarmee wordt gecommuniceerd.
Een kloksnelheid van 100 kHz betekent dat een klokperiode 10 µs duurt. Dit betekent echter niet dat er 100.000 gegevensbits per seconde beschikbaar zijn: ook adresbits, bevestigingsbits en andere protocolinformatie kosten tijd.

[2] `0x40` is een hexadecimaal getal (grondtal 16) en komt overeen met $64 = 4 \cdot 16^1 + 0 \cdot 16^0$ in het decimale stelsel en `1000000` in het binaire stelsel (7 bits).

[3] Een bus is een verzameling gedeelde elektrische verbindingen waarmee meerdere elektronische componenten kunnen communiceren. Bij I²C bestaat de communicatiebus uit twee signaallijnen: SDA (gegevens) en SCL (kloksignaal).

Aandachtspunten bij het aansluiten

Bij het aansluiten van I²C-apparaten moeten we rekening houden met het volgende:

- SDA van alle apparaten wordt met dezelfde SDA-lijn verbonden.
- SCL van alle apparaten wordt met dezelfde SCL-lijn verbonden.
- De apparaten delen een gemeenschappelijke GND.
- De pull-upspanning moet geschikt zijn voor alle aangesloten apparaten.
- Meerdere modules kunnen al pull-upweerstanden bevatten. Deze komen dan parallel te staan.
- Apparaten op dezelfde bus mogen geen conflicterende adressen hebben.

De GPIO-pinnen van de Raspberry Pi Pico werken op 3,3 V en zijn niet 5 V-tolerant. De I²C-signaallijnen mogen daarom niet rechtstreeks naar 5 V worden opgetrokken.

Appendix D: Afleiding bewegingsvergelijking

Bewegingsvergelijking van het aeropendulum (open loop)

We beschouwen een aeropendulum bestaande uit een roterende arm met een propeller. De propeller levert een stuwkracht waardoor de arm om zijn draaipunt roteert.
Volgens de tweede wet van Newton voor rotaties geldt:

$$\sum \tau = J\ddot{\theta}. \tag{16}$$

Hierin is J het traagheidsmoment rond het draaipunt en θ de hoek van de arm ten opzichte van de verticale evenwichtspositie.

Op de arm werken drie koppels:

- Het motorkoppel door de stuwkracht van de propeller.
- Het zwaartekrachtsmoment.
- Het dempende koppel door wrijving.

Het motorkoppel schrijven we als $\tau_m = l_T F_T$, waarbij l_T de afstand tussen het draaipunt en de aangrijpingslijn van de stuwkracht is. Het zwaartekrachtsmoment is: $\tau_g = -mgl\sin(\theta)$. Hierin is l de afstand van het draaipunt tot het massamiddelpunt. We modelleren de wrijving als een krachtmoment dat evenredig is met de hoeksnelheid: $\tau_w = -b\dot{\theta}$.
De bewegingsvergelijking wordt daarmee:

$$J\ddot{\theta} = l_T F_T - b\dot{\theta} - mgl\sin(\theta) \tag{17}$$

Of, na herschikken:

$$J\ddot{\theta} + b\dot{\theta} + mgl\sin(\theta) = l_T F_T \tag{18}$$

Relatie tussen PWM en stuwkracht

We nemen aan dat de stuwkracht direct en lineair afhangt van de PWM-waarde u:

$$F_T = k_u u \tag{19}$$

[4].
Hierin is k_u een evenredigheidsconstante. We defineren nu $A = l_T k_u$, zodat het motorkoppel kan worden geschreven als $\tau_m = Au$. Daarmee wordt de bewegingsvergelijking:

$$\boxed{J\ddot{\theta} + b\dot{\theta} + mgl\sin(\theta) = Au} \tag{20}$$

Dit is een niet-lineaire differentiaalvergelijking van de tweede orde.

Open-loop-evenwicht

Bij open-loop-aansturing wordt de PWM-waarde u ingesteld zonder terugkoppeling van de gemeten hoek. We beschouwen een constante PWM-waarde u_0, waardoor ook het motorkoppel Au_0 constant is.
Als het systeem een stabiel evenwicht bereikt, verandert de hoek niet meer in de tijd. In deze stationaire toestand gelden:

$$\dot{\theta} = 0, \quad \ddot{\theta} = 0 \tag{21}$$

[4] De stuwkracht is in werkelijkheid een niet-lineaire functie van de PWM-waarde. Voor kleine afwijkingen rond het evenwicht kan deze relatie echter worden benaderd als lineair.

De bewegingsvergelijking vereenvoudigt hierdoor tot:

$$mgl\sin(\theta_0) = Au_0 \tag{22}$$

Hierin is θ_0 de evenwichtshoek die hoort bij de ingestelde PWM-waarde u_0.
Door de vergelijking op te lossen naar de evenwichtshoek vinden we:

$$\boxed{\theta_0 = \arcsin\left(\tfrac{Au_0}{mgl}\right)} \tag{23}$$

Deze uitdrukking beschrijft de evenwichtshoek op de tak $-\frac{\pi}{2} \leq \theta_0 \leq \frac{\pi}{2}$ ($-90° \leq \theta_0 \leq 90°$). Een dergelijk evenwicht bestaat alleen wanneer:

$$|Au_0| \leq mgl$$

Wanneer het motorkoppel groter wordt dan dit maximale zwaartekrachtsmoment,

$$|Au_0| > mgl \tag{24}$$

kan de zwaartekracht het motorkoppel niet meer compenseren. Er bestaat dan geen statische evenwichtshoek meer. De arm blijft in de richting van het motorkoppel versnellen zolang de hoeksnelheid voldoende klein is.